

10 Proven Memory Prompts

Copy any prompt, fill the [BRACKETS], send. Mined from 500+ hours of top Claude builders.

1

Build a Lean CLAUDE.md

CLAUDE.md is injected into every conversation as a system prompt; a lean, well-structured file directly improves both cost and output quality.

Help me create a CLAUDE.md file for this project. Ask me clarifying questions first, then keep the file lean – under 150–200 lines. Structure it as the WHAT (tech stack, project structure, build commands), the WHY (purpose of each part), and the HOW (how I want you to work). It's read on every single message, so every line must earn its place – no history, no filler.

2

CLAUDE.md as an Index, Not a Dump

Keeps the always-loaded system prompt small while preserving deep context — Claude reads detail files only when relevant, a major token saving.

Restructure my CLAUDE.md so it routes to other files instead of containing everything. It should only hold the never-forget rules plus a map: 'For who I am, read me.md. For business details, read business.md. For project status, read current-priorities.md.' Move all the detailed content into those separate files so you only load detail when it's relevant.

3

The 95% Confidence Rule

Nate puts this in every CLAUDE.md — it prevents the single biggest source of token waste by forcing alignment before any code is written.

Add this rule to my CLAUDE.md word for word: 'Do not make any changes until you have 95% confidence in what you need to build. Ask follow-up questions one at a time until you reach that confidence level.' Then read it back to me so I can confirm it's in there.

4

Self-Evolving Applied Learning Block

Makes the project's memory learn over time — but the one-short-bullet cap means it never bloats into context pollution.

Add an 'Applied Learning' section to the bottom of my CLAUDE.md with this instruction: 'When something fails repeatedly, when I have to re-explain something, or when a workaround is found for a tool limitation, add a one-line bullet here. Keep each bullet under 15 words, no explanations. Only add things that will save time in future sessions.'

5

Memory Consolidation (Dream)

Claude Code's auto-memory accumulates duplicates, contradictions and stale data over time; a periodic consolidation pass keeps the index tight and current.

Do a reflective consolidation pass over my memory files: merge any duplicate files, resolve contradictions, convert relative dates like 'next Friday' into absolute dates, and synthesize recent sessions into durable, well-organized memories. Keep the memory index under 200 lines – it's an index, not a dump. Link to memory files with one-line descriptions; do not copy full content into it.

6

Turn Corrections Into Permanent Memory

Turns every correction into a permanent improvement so the project gets smarter over time instead of repeating the same errors.

That second version was much better than your first attempt. Now update my CLAUDE.md (or the relevant skill) so this preference is captured permanently – here's what I told you to fix, here's what I liked, here's what I didn't like. Reflect all of it so you never make that mistake again.

7

Karpathy Markdown Wiki (Second Brain)

Replicates Karpathy's no-vector-DB RAG — a clever folder structure gives a cheap, human-readable traversal path through thousands of notes; one X user cut query tokens ~95%.

I want a personal knowledge base with no vector database – just markdown files you organize. Set up a vault with a 'raw' folder for sources I dump in, a 'wiki' folder of interlinked markdown articles you maintain, and a master index.md. Create a CLAUDE.md that describes this structure. When I drop a document in 'raw' and say 'ingest that', file it into the wiki and update related pages.

8

Session Handoff Document

Resets context without losing it — Claude can re-read project files itself, so you only need to carry forward the conversational state.

Summarize this conversation succinctly into a handoff.md file: where we started, the decisions we locked in, what shipped, the key files for next session, the running state, what's left, and any open questions – written so a new agent could pick up exactly where you left off. Then I'll /clear and start fresh from that file.

9

Personal / Business Context File

Without context of the problem you solve, Claude can't solve it well; a referenced context file personalizes every output without you re-explaining yourself.

Help me create a business.md file covering: what my business is, who I serve, how it makes money, my target customer, the 3 biggest problems I solve, my revenue, my channels, and my goals. Ask me one question at a time to fill it in. Then add a rule to CLAUDE.md: 'When I ask anything strategic or growth-related, read business.md first for context.'

10

Decision Log

An auditable decision history means every architectural call is a paragraph you never have to re-type or re-explain in a future session.

From now on, whenever we make a meaningful architectural or strategic decision, log it to `decisions/log.md` with the date, the decision, the reasoning, and the context. Save decisions, not conversations. Create that file now and add the decisions we've already made today.

11

GSD Multi-File State System

Splitting memory into purpose-built files (the GSD framework) means Claude always knows the goal, the spec, the plan, and the live progress — without re-deriving them from a bloated single file.

Set up a four-file state system for this project so you always know where things stand, instead of one `CLAUDE.md` doing everything. Create: `project.md` (the North Star — what we're building and why), `requirements.md` (exactly what to build), `roadmap.md` (phases, what's done, what's next), and `state.md` (current progress and where we left off). At the start of every session read all four; at the end of a work block, update `state.md` and `roadmap.md`.

12

Voice Memory From Your Real Posts

Anyone can make AI content — making it authentically sound like you is the differentiator; a referenced voice file makes every generated draft match your real style.

I'm pasting in a batch of content I've personally written: [PASTE 15–30 OF YOUR REAL POSTS, EMAILS, OR CAPTIONS]. Study them and create a `voice.md` file written as direct INSTRUCTIONS for emulating my style — vocabulary, sentence length, punctuation habits, rhetorical patterns, what I never do. Include good and bad examples. Then add a rule to `CLAUDE.md`: 'When writing anything in my voice, read `voice.md` first.'

13

Caveman Conciseness Rule

March 2025 research found forcing brief responses can improve large-model accuracy by combating RLHF-trained overthinking — plus a real token saving on every reply.

Add a conciseness instruction to my `CLAUDE.md`: 'Be concise. No filler, no preamble, no flattery — straight to the point, fewer words. When I ask a question, answer it directly first, then add detail only if needed.' Confirm it's in there.

14

Local Docs Index for a Big Knowledge Base

A tiny routing index turns a huge knowledge base into a cheap lookup — Claude jumps to the right section instead of burning context re-scanning the whole thing.

I'm going to work with [BIG DOCS SET — e.g. the Claude Code docs, an API reference, a long PDF]. Read it all once, then build me a lean `docs-index.md`: a small file that lists each topic and the exact file path or URL section where the answer lives. From now on, when I ask about this, check the index first and jump straight to the right section instead of re-reading everything.

15

Stateless Automation Memory Trail

Scheduled and cloud automations are stateless; a single overwritten status file gives them just enough memory to improve run-over-run without log bloat.

This task will run on a schedule, and each run starts with a blank slate. Give it a memory trail: have it read `last-run.md` at the start, do its job, then overwrite `last-run.md` at the end with the date, what it did, the current state, and any known issues for the next run. Use a single overwritten file, not an append log, so it never bloats. Create `last-run.md` now.